Software Requirements Specification — Custom Intrusion Detection System & Packet Analyzer

Sep 30, 2026 · @Satyam

This SRS specifies what the system must do and how well, following the IEEE 830 / ISO/IEC/IEEE 29148 structure. It reflects the system as implemented in the project repository.

Name	Enrollment No.
Aditya Pratap Singh Bhadoria	24114005
Aditya Ranjan	24114006
Daddi Om Santosh	24114028
Deokar Parth Rajesh	24114032
Kothawade Manthan	24114050
Satyam Sharma	24114088

1. Introduction

1.1 Purpose

This document specifies the functional and non-functional requirements of a network Intrusion Detection System (IDS) and packet analyzer built as the Computer Networks semester project. It is written for the project team, the course instructor and evaluators, and anyone extending the system.

1.2 Scope

The system captures live traffic from a network interface (or replays a saved capture file), decodes every protocol layer from raw bytes, and raises alerts when traffic matches configurable rules or behavioural detectors. It correlates related alerts into incidents, writes a structured alert log, and shows a real-time terminal dashboard.

The system observes and alerts only. It does not block, drop, modify or inject traffic, and it does not decrypt TLS.

1.3 Definitions and abbreviations

Term	Meaning
IDS	Intrusion Detection System: monitors traffic and reports suspicious activity
Alert	One detection event raised by a rule or detector
Summary	A single record replacing repeated alerts of the same rule and source within the deduplication window
Incident	A group of related alerts correlated across attack stages
Rule	One line in the rule file describing what to match and how to report it
Detector	Built-in stateful logic invoked by a rule (e.g. SYN flood, port scan)
Scenario	A generated capture plus the alerts it must and must not produce, used as a regression test
Engine time	The clock used for detection, taken from packet timestamps, never from the wall clock
pcap / pcapng	Standard capture file formats
BPF	Berkeley Packet Filter expression used to pre-filter captured traffic
JSONL	JSON Lines: one JSON record per line
eTLD+1	Registered domain per the Public Suffix List (e.g. example.co.uk)
JA3	Fingerprint of a TLS client computed from its ClientHello
GRO / TSO	Receive and transmit offloads that merge or split segments around the capture point

1.4 References

- IEEE Std 830-1998 and ISO/IEC/IEEE 29148:2018, requirements specification structure.
- RFC 791 (IPv4), RFC 8200 (IPv6), RFC 9293 (TCP), RFC 768 (UDP), RFC 792 / RFC 4443 (ICMP, ICMPv6), RFC 826 (ARP).

- RFC 1034 / RFC 1035 (DNS), RFC 9110 / RFC 9112 (HTTP), RFC 959 (FTP), RFC 8446 (TLS 1.3), RFC 8701 (GREASE).
- RFC 1858 and RFC 7112 (tiny-fragment attacks), RFC 5227 (ARP probes), RFC 1982 (serial number arithmetic).
- Ptacek & Newsham, *Insertion, Evasion, and Denial of Service: Eluding Network Intrusion Detection*, 1998.
- Project repository documentation: README, docs/ARCHITECTURE.md, docs/RULES.md, docs/SCENARIOS.md, docs/ADDING_A_DETECTOR.md.
- Project proposal submitted earlier for this course.

1.5 Document overview

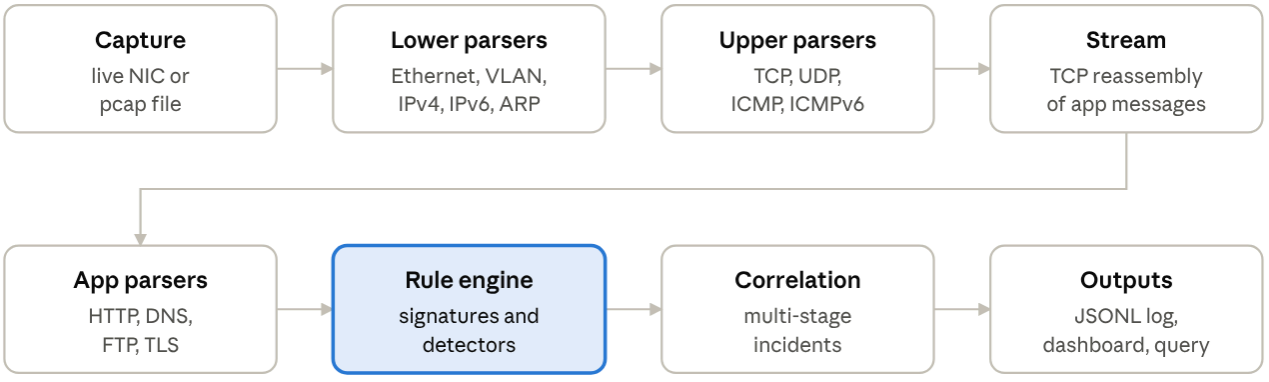
Section 2 describes the system as a whole. Section 3 covers external interfaces. Sections 4 to 7 list functional requirements, each with an identifier (e.g. FR-CAP-1) used for traceability. Section 8 gives non-functional requirements with measured values. Section 9 lists constraints and known limitations. Section 10 maps requirements to verification and to the course syllabus.

2. Overall Description

2.1 Product perspective

The system is a self-contained command-line application for Linux. It sits passively on a host or a monitored network segment and processes every packet through a fixed pipeline of stages. Each stage reads and writes one shared per-packet record, so stages can be built, tested and replaced independently.

Every packet passes through eight stages in a fixed order



Capture runs in its own goroutine; every later stage runs in one ordered goroutine.

processing pipeline · 8 stages

Capture runs in its own goroutine. All parsing, reassembly, detection and correlation run in one ordered goroutine, so packets are never reordered before stateful detectors see them.

2.2 Product functions (summary)

- Capture traffic from a live interface with an optional BPF filter, or replay pcap/pcapng files.
- Decode Ethernet, 802.1Q/QinQ VLAN, ARP, IPv4, IPv6 with extension headers, TCP, UDP, ICMP/ICMPv6, DNS, HTTP/1.x, FTP control and TLS ClientHello from raw bytes.
- Reassemble TCP application messages for the protocols it parses.
- Match packets against a Snort-like rule language and run built-in stateful detectors.
- Detect scanning, flooding, spoofing, fragmentation abuse, web attacks, DNS abuse, tunnelling, beaconing, threat-intelligence hits and traffic anomalies.
- Deduplicate repeated alerts and correlate related alerts into multi-stage incidents.
- Log alerts, incidents, statistics and events as JSON Lines; query the log; show a live dashboard.

2.3 User classes

User	Needs	Technical level
Network / security analyst	Watch the dashboard, investigate alerts and incidents, query the log	High
Rule author	Write and tune rules, thresholds, variables and feeds without recompiling	High
Developer (project team)	Add parsers and detectors following the documented contract and scenario harness	High
Evaluator / instructor	Run the demo, replay captures, inspect results	Medium

2.4 Operating environment

- Linux (kernel with packet sockets); developed on Ubuntu under WSL2 and intended for demonstration on native Linux or a VM with bridged/host-only networking.
- Go 1.27.x toolchain; libpcap runtime and development headers (libpcap-dev).
- A terminal with colour support for the dashboard.
- Live capture requires the CAP_NET_RAW and CAP_NET_ADMIN capabilities on the binary; replaying files needs no privileges.

2.5 Design and implementation constraints

- Language: Go. Protocol decoding must be hand-written from raw bytes; third-party packet decoders may be used only as test oracles.
- Packet capture through libpcap only for obtaining raw frames.
- Link type: Ethernet only (DLT_EN10MB); other link types are rejected with a clear error.
- Parsers must be stateless; all per-connection state lives in the stream layer and the rule engine.
- Detection must use packet time, so replaying the same capture always produces the same alerts.

2.6 Assumptions and dependencies

- The monitored traffic is traffic the team owns or is authorised to monitor.
- The capture point sees both directions of a conversation for handshake-, reply- and stream-based detections.
- Threat-intelligence feeds are downloaded separately by the operator and are only as current as their last download.

3. External Interface Requirements

3.1 Command-line interface

Command	Purpose	Key options
`ids run` (default)	Capture, detect, log and display	`-i IFACE` or `-r FILE`, `-f BPF`, `-rules PATH`, `-whitelist CIDRs`, `-log PATH`, `-log-max-size`, `-log-max-files`, `-no-tui`, `-stats-interval`, `-stream-ports`, `-stream-max-mem`, `-pprof 127.0.0.1:PORT`
`ids query`	Search the alert log	`-log`, `-severity`, `-since` or `-from`/`-to`, `-src`, `-dst`, `-sid`, `-category`, `-kind`, `-type`, `-follow`, `-json`
`capturedump`	Debug tool: one decoded line per frame	`-i`, `-r`, `-f`, `-rules`, `-alerts-only`

- **IF-CLI-1** Exit codes shall be 0 on clean shutdown, 1 on rule or capture errors, 2 on usage errors.

- **IF-CLI-2** When standard output is not a terminal, `ids run` shall switch to line-per-alert output automatically.
- **IF-CLI-3** SIGHUP shall reload rules and feeds; SIGINT/SIGTERM shall shut down gracefully.

3.2 Dashboard (terminal UI)

- **IF-UI-1** Panels: header (source, uptime, rules, reload result, feed ages, baseline state), incidents, traffic rates, capture health, protocol breakdown, top talkers, top alerting sources, recent alerts.
- **IF-UI-2** Keys: `q` quit, `p` pause the alert feed, `r` reload rules, `i` toggle incidents / raw alerts.
- **IF-UI-3** Any non-zero drop counter and any stale feed shall be shown in red; alerts shall be coloured by severity.
- **IF-UI-4** The dashboard shall refresh at least once per second and shall never block the processing pipeline.

3.3 Files

File	Format	Direction
Rule file (`rules.conf`)	Snort-like text, one rule or directive per line	Input
Threat feeds (`feeds/*.txt`)	One IP/CIDR, domain or JA3 hash per line	Input
Capture files	pcap, pcapng (Ethernet link type)	Input
Alert log (`ids-alerts.jsonl`)	JSON Lines, mode 0600, size-rotated	Output

Log record types: `alert` (kinds alert, summary, incident, incident_update), `stats` (periodic and shutdown counters), `event` (rule reloads, feed warnings, baseline state).

3.4 Software interfaces

- libpcap via the gopacket/gopacket Go bindings, used only to open interfaces or files, apply BPF filters and read raw frames.
- golang.org/x/net/publicsuffix for registered-domain calculation.
- rivo/tview for the terminal UI.

3.5 Hardware and communication interfaces

- Any Ethernet-framed network interface visible to the host, including the Linux loopback interface.

- The system opens no network listeners, except an optional diagnostics endpoint bound only to 127.0.0.1.

4. Functional Requirements: Capture, Parsing and Reassembly

4.1 Packet capture (FR-CAP)

ID	Requirement
FR-CAP-1	The system shall capture every frame on a chosen interface in promiscuous mode, or read frames from a pcap/pcapng file.
FR-CAP-2	The system shall accept an optional BPF filter and report the libpcap error for an invalid one.
FR-CAP-3	Each frame shall carry its capture timestamp, captured length and wire length, and an owned copy of its bytes.
FR-CAP-4	Live capture shall never block on a slow consumer: frames that overflow the internal queue are dropped and counted. File input shall wait for the consumer and never drop.
FR-CAP-5	The system shall report frames captured, kernel drops, queue drops and queue depth.
FR-CAP-6	The system shall reject non-Ethernet link types, missing interfaces (listing the available ones), missing permissions (naming the setcap fix) and damaged capture files with a clear error.

4.2 Lower-layer decoding (FR-L2L3)

ID	Requirement
FR-L2L3-1	Decode Ethernet source/destination MAC and EtherType; treat values below 0x0600 as 802.3 length frames.
FR-L2L3-2	Skip stacked 802.1Q / 802.1ad VLAN tags, with a cap on the number walked.

ID	Requirement
FR-L2L3-3	Decode IPv4 fields including options, fragmentation fields and TTL; verify the header checksum.
FR-L2L3-4	Decode IPv6 and walk the extension header chain (Hop-by-Hop, Routing, Fragment, Destination Options, AH, ESP, No Next Header) with a cap.
FR-L2L3-5	Decode Ethernet/IPv4 ARP requests and replies.
FR-L2L3-6	Bound all later parsing by the IP total length so Ethernet padding is never read as data.
FR-L2L3-7	Oversized frames produced by GRO/TSO or BIG TCP shall not be reported as malformed.

4.3 Transport-layer decoding (FR-L4)

ID	Requirement
FR-L4-1	Decode TCP ports, sequence and acknowledgement numbers, all flags, window and options, and locate the payload by the data offset.
FR-L4-2	Decode UDP ports and length, and bound the payload by the UDP length.
FR-L4-3	Decode ICMP and ICMPv6 type and code with human-readable labels, echo identifier and sequence, and the original header quoted inside ICMP error messages.
FR-L4-4	Classify the L4 checksum as valid, invalid or not checkable (fragments, truncated captures, merged segments), using the IPv4/IPv6 pseudo-header. A zero UDP checksum is "not used" over IPv4 and invalid over IPv6.

4.4 TCP stream reassembly (FR-STR)

ID	Requirement
FR-STR-1	Reassemble application messages for configurable TCP ports (default 21, 53, 80, 443, 8000, 8080, 8443), buffering only up to message boundaries: HTTP headers, one DNS-over-TCP message, one FTP line, one TLS handshake record.
FR-STR-2	Use wraparound-safe sequence arithmetic; reorder out-of-order segments within a bounded window; resynchronise at the next clean message boundary after a gap or a mid-stream start.
FR-STR-3	Ignore identical retransmissions; flag overlapping segments with conflicting data as an evasion attempt without choosing either copy.
FR-STR-4	Create stream state only for flows that carry data, and enforce per-flow and global memory caps with least-recently-used eviction.

4.5 Application-layer decoding (FR-APP)

ID	Requirement
FR-APP-1	Identify the protocol by content, using ports only as a hint.
FR-APP-2	DNS: decode header counts, question name/type/class with compression pointers, over UDP and TCP; flag malformed names (loops, label > 63, name > 255, reserved label types).
FR-APP-3	HTTP/1.x: decode request/status line, key headers, the raw header block, query string, and the URI percent-decoded (including overlong UTF-8); handle pipelined requests.
FR-APP-4	FTP: decode commands and reply codes, including multi-line replies.
FR-APP-5	TLS: extract SNI from the ClientHello and compute the JA3 fingerprint with GREASE values removed.

ID	Requirement
FR-APP-6	Credentials (HTTP Authorization values, FTP PASS arguments) shall never be stored, logged or displayed; only their presence is recorded.

5. Functional Requirements: Rule Language and Detection Engine

5.1 Rule language (FR-RULE)

A rule has the form `action proto src_addr src_port direction dst_addr dst_port (options)`, for example:

```
alert tcp $EXTERNAL_NET any -> $HOME_NET any (msg:"Possible SYN flood";
detect:syn_flood; track:by_dst; count:100; seconds:5; severity:high;
category:dos; sid:1000001; rev:2;)
```

ID	Requirement
FR-RULE-1	Actions `alert` and `pass`; protocols ip, tcp, udp, icmp, arp; directions `->` and `<>`.
FR-RULE-2	Addresses and ports: any, single values, CIDRs, ranges, lists, negation, and variables (`var NAME value`, `$NAME`), including name-list variables.
FR-RULE-3	Header options: flags, ttl, itype/icode, same_ip, same_port, eth_dst (broadcast, multicast, zero, negatable), arp_op, dsize.
FR-RULE-4	Content options: content (with hex escapes and nocase), app_proto, app_field, app_content, app_domain, app_reason, regex (RE2, input capped at 16 KiB), stream_anomaly.
FR-RULE-5	Threshold option detection_filter (count N in S seconds, by source or destination).
FR-RULE-6	Detector invocation via `detect:NAME` with detector-specific options (kind, track, count, seconds, ratios).
FR-RULE-7	Directives: `var`, `arpbind IP MAC`, `feed TYPE NAME PATH`, `stage NAME CATEGORIES`.

ID	Requirement
FR-RULE-8	Every rule needs msg and a unique sid; optional rev, severity (low, medium, high, critical) and category.
FR-RULE-9	Loading shall report every error with file:line and reason, reject unknown options and typos, and never silently ignore a line.

5.2 Detection engine (FR-ENG)

ID	Requirement
FR-ENG-1	The engine clock shall be the maximum packet timestamp seen; packets with earlier timestamps are treated as zero elapsed time.
FR-ENG-2	Replaying the same capture shall produce byte-identical alert output.
FR-ENG-3	Sliding windows shall be exact (bounded timestamp deques) for counts, and sub-bucketed (at most 10% boundary error) for byte volumes.
FR-ENG-4	Every stateful table shall have a key cap with least-recently-used eviction and counted evictions.
FR-ENG-5	Pass rules and the source whitelist shall suppress matching alerts.
FR-ENG-6	Repeated matches of one rule for one tracked address within the deduplication window (default 60 s) shall produce one alert followed by one summary with the total count.
FR-ENG-7	Rules and feeds shall reload atomically; a failed reload keeps the previous rule set and reports all errors.
FR-ENG-8	A bad L4 checksum alone shall never raise an alert (checksum offload and receive merging produce false ones).
FR-ENG-9	Every alert shall carry time, sid, rev, message, severity, category, protocol, addresses, ports, count, kind, attribution (reliable or spoofable) and detector-specific details.

5.3 Threat intelligence (FR-TI)

ID	Requirement
FR-TI-1	Load IP/CIDR, domain and JA3 feeds from files, with fast lookup for 100k+ entries.

ID	Requirement
FR-TI-2	Reject private, reserved, loopback, link-local, CGNAT, multicast and unspecified entries with a warning.
FR-TI-3	Match domains against DNS query names, TLS SNI and HTTP Host, including subdomains.
FR-TI-4	Track each feed's age and warn (log event, red on the dashboard) when older than its maximum age (default 7 days).

6. Functional Requirements: Detections Catalogue (FR-DET)

The system shall detect each threat below with the default rule set. Thresholds are defaults in `rules.conf` and are configurable. Every detection has a scenario that must fire and a look-alike that must not.

ID	Threat	Detection condition (default)	Must not fire on	Severity
FR-DET-1	SYN flood	⩾ 100 incomplete handshakes in 5 s per source or per victim, incomplete ratio ⩾ 0.8, ⩽ 5 distinct ports	Busy NAT sources with completed handshakes; scans	High
FR-DET-2	Port scan (SYN, FIN, NULL, Xmas, UDP)	⩾ 20 distinct probed ports from one source in 10 s; UDP probes seen via ICMP port unreachable	Completed connections; FINs from connections open before start	High
FR-DET-3	Stealth packets	NULL, Xmas or SYN+FIN flags (single packet)	Any normal TCP	High
FR-DET-4	Host sweep / ping sweep	Same port probed on ⩾ 15 hosts in 30 s; echo requests to ⩾ 15 hosts in 30 s	Browsing many hosts on 443	Medium

ID	Threat	Detection condition (default)	Must not fire on	Severity
FR-DET-5	UDP flood	⩾ 10,000 packets (or 100 MB) in 5 s with reply ratio ⩽ 0.02	QUIC downloads, VoIP, UDP request/response	High
FR-DET-6	ICMP floods	⩾ 1,000 echo requests in 5 s; ⩾ 100 unsolicited echo replies (Smurf victim); ⩾ 1,000 ICMP errors	Monitoring pings	High / Medium
FR-DET-7	Reflection flood	⩾ 2,000 packets in 5 s to one host from reflector ports (DNS, NTP, SSDP, memcached, etc.)	Normal DNS/NTP use	High
FR-DET-8	Land attack	Source IP and port equal destination on a real (non-loopback) frame	Loopback self-traffic	High
FR-DET-9	Smurf amplifier	Echo request to Ethernet broadcast or 255.255.255.255	IPv6 multicast discovery (reported low)	High
FR-DET-10	Fragment attacks	Overlapping fragments (teardrop), tiny first fragments, offset + length > 65,535 (Ping of Death), incomplete-datagram floods	Exact duplicate fragments; normal large pings	High
FR-DET-11	TTL anomaly / spoofing	Hop distance differs from an established baseline by > 3, 5 times in 60 s, flows without a completed handshake	Mixed OSes behind NAT; load-balanced servers	Low

ID	Threat	Detection condition (default)	Must not fire on	Severity
FR-DET-12	Traceroute / low TTL	⩾ 5 Time Exceeded to one host in 10 s; TTL < 3 from outside (excluding link-local multicast)	MLD/IGMP	Low
FR-DET-13	ARP spoofing	Static binding violation, MAC flip-flop, unsolicited replies, one MAC claiming many IPs, Ethernet/ARP MAC mismatch, invalid sender MAC	ARP probes from 0.0.0.0; boot announcements; one-off DHCP reassignment (low)	Critical–Low
FR-DET-14	Slowloris family	⩾ 20 flows with incomplete headers > 10 s old; slow request bodies; sustained zero windows	Idle browser keep-alive; normal uploads	High
FR-DET-15	TCP evasion	Overlapping segments with conflicting data; oversized headers	Identical retransmissions	High / Medium
FR-DET-16	Web attacks	SQL injection, XSS, path traversal, overlong UTF-8, command injection, Log4Shell (incl. obfuscated), Shellshock, scanner User-Agents, double encoding revealing traversal	Searches containing SQL words; encoded URLs in ad/analytics parameters	Critical–Medium
FR-DET-17	Cleartext credentials and FTP brute force	HTTP Basic auth, FTP PASS; ⩾ 5 FTP 530 replies in 30 s	–	Medium / High
FR-DET-18	DNS abuse	AXFR and repeated ANY queries; malformed messages	Normal queries	Medium

ID	Threat	Detection condition (default)	Must not fire on	Severity
FR-DET-19	DNS spoofing	⩾ 10 unsolicited responses in 10 s; ⩾ 10 distinct IDs racing for a pending query in 2 s (Kaminsky); question mismatch	A single unmatched response	Critical–High
FR-DET-20	DNS amplification	⩾ 1 MB of responses in 5 s at ⩾ 10× query bytes, or unrequested	Normal resolution	High
FR-DET-21	DNS tunnelling	⩾ 50 unique subdomains of one registered domain in 60 s with high entropy or long names; ⩾ 100 TXT/NULL queries	CDN subdomains under multi-part suffixes (co.uk)	High
FR-DET-22	DGA malware	⩾ 30 NXDOMAIN for distinct, mostly high-entropy names in 60 s	Typos, broken search domains	Medium
FR-DET-23	Encrypted DNS bypass	TLS SNI of known DoH resolvers; data on TCP 853	Scan probes to 853	Low
FR-DET-24	ICMP tunnelling	⩾ 10 non-standard echo payloads in 60 s with varied sizes or entropy > 6 bits/byte	Standard Linux/Windows ping payloads	Medium
FR-DET-25	Threat intelligence	Traffic to or from listed IPs, domains or JA3 fingerprints	Private-range feed entries (rejected)	High / Critical
FR-DET-26	Beaconing (C2)	Regular connection starts from an internal host (median interval 10 s–1 h, jitter- and skip-tolerant, persistence required)	Random-interval browsing; single long connections; NTP	Medium

ID	Threat	Detection condition (default)	Must not fire on	Severity
FR-DET-27	Traffic anomaly	A metric (packets, bytes, connections, DNS, destinations, per-host fan-out) > 4 robust deviations above baseline for 3 intervals after learning, above a minimum level	Spikes during learning; single-interval spikes	Medium

7. Functional Requirements: Correlation, Logging, Dashboard and Query

7.1 Correlation (FR-COR)

Alert categories map to ordered kill-chain stages through `stage` directives: recon → exploit → c2 → exfil, with impact (dos) separate.

ID	Requirement
FR-COR-1	Classify every alert's source as reliable (completed TCP handshake, application layer, stream layer, MAC-keyed ARP, beacon) or spoofable (UDP/ICMP sources, incomplete handshakes).
FR-COR-2	A spoofable source shall never be named as an attacker in an incident.
FR-COR-3	Multi-stage incident: one reliable attacker reaching ≥ 2 distinct stages against the same victim, in stage order, within 24 h after recon and 1 h between later stages (high; critical at 3+ stages).
FR-COR-4	Compromised host: a host that was the victim of an exploit-stage alert and later the reliable source of a c2 or exfil alert within 24 h (critical).
FR-COR-5	Callback: after an exploit-stage alert from X to Y, Y opens a new TCP connection to X within 120 s (critical).

ID	Requirement
FR-COR-6	An incident is emitted once when created, then updated only when it gains a stage or rises in severity.
FR-COR-7	Entity state is bounded (64 alerts per entity, 10,000 entities, 24 h expiry).

7.2 Logging (FR-LOG)

ID	Requirement
FR-LOG-1	Write alerts, incidents, statistics and events as JSON Lines, one record per line, file mode 0600.
FR-LOG-2	Rotate by size (default 100 MB, 5 backups); a file never exceeds the limit.
FR-LOG-3	The writer shall never block the pipeline; records it cannot accept are dropped and counted.
FR-LOG-4	On shutdown, flush pending summaries, write a final statistics record, drain and fsync the log.

7.3 Dashboard (FR-UI)

ID	Requirement
FR-UI-1	Show packets/s and bytes/s (current and 60 s average), protocol and application breakdown, top talkers, top alerting sources, recent alerts (last 200) and active incidents.
FR-UI-2	Show capture, queue, log and stream health counters, baseline state and feed ages.
FR-UI-3	Receive alerts through a bounded, dropping channel; pausing the feed shall not stall processing.

7.4 Log query (FR-QRY)

ID	Requirement
FR-QRY-1	Filter by minimum severity, time range, source/destination IP or CIDR, sid, category, kind and record type.
FR-QRY-2	Read the current and rotated files in time order.
FR-QRY-3	Skip a partially written last line and count, rather than fail on, invalid lines.
FR-QRY-4	Follow the log live across rotations; output as a table or raw JSON Lines.

8. Non-Functional Requirements

Measured values come from the project's benchmark, replay and soak tests on the development machine (WSL2).

ID	Attribute	Requirement	Measured
NFR-PERF-1	Throughput, mixed traffic	⩾ 250,000 packets/s end to end with the full rule set	~300,000 packets/s (1M-packet replay, 74 rules)
NFR-PERF-2	Throughput, worst case	⩾ 150,000 packets/s on a 1M-packet spoofed SYN flood	~185,000 packets/s
NFR-PERF-3	Rule engine alone	⩾ 200,000 packets/s with 30 rules	1.2–2.2 million packets/s, 0 allocations per packet
NFR-PERF-4	Alert latency	Alerts displayed within 1 s of the triggering packet's arrival	⩽ ~100 ms capture batching + 1 s refresh
NFR-CAP-1	Capture loss	0 kernel and queue drops at lab traffic rates, including 12,000-connection bursts	0 drops (32 MB ring, immediate mode off)
NFR-MEM-1	Bounded memory	Peak memory ⩽ 256 MB under 1 million spoofed sources	182 MB

ID	Attribute	Requirement	Measured
NFR-MEM-2	Stream memory	Reassembly memory ⩽ 64 MiB (configurable)	Enforced with LRU eviction
NFR-REL-1	Stability	60-minute soak: flat memory and goroutine count, every attack burst alerted, clean exit	Pass (goroutines constant at 10)
NFR-REL-2	Robustness	No crash on any input bytes; fuzzing of every parser, the rule loader and the full pipeline	Millions of fuzz executions per target, no failures
NFR-REL-3	Determinism	Identical alerts on every replay of the same capture	Pass
NFR-ACC-1	False positives	0 false-positive alerts on the benign public capture corpus and own recordings	0 on ~80 public captures + 20 min of own traffic
NFR-SEC-1	Credential privacy	No credential value in any log, output or error	Verified by redaction tests on every case
NFR-SEC-2	Log protection	Alert logs created with mode 0600	Pass
NFR-SEC-3	Least privilege	Live capture via file capabilities, not root; diagnostics bound to 127.0.0.1 only	Pass
NFR-SEC-4	Safe matching	Regular expressions use RE2 (linear time) with capped input to prevent ReDoS	Pass
NFR-MNT-1	Maintainability	Stages communicate only through the shared packet record; new detectors follow docs/ADDING_A_DETECTOR.md with a positive and a look-alike scenario	100+ scenarios in the harness
NFR-MNT-2	Test quality	Each change passes a mutation check: deliberately broken logic must fail a test	All mutants killed per stage
NFR-MNT-3	Continuous integration	Format, vet, static analysis, race-enabled tests, scenarios and a short fuzz run on every push	GitHub Actions workflow

ID	Attribute	Requirement	Measured
NFR-PORT-1	Portability	Runs on Linux amd64 with libpcap; no root needed for file replay	Pass
NFR-USE-1	Usability	Every configuration error names file, line and reason; every capture error names the fix	Pass

9. Constraints, Assumptions and Known Limitations

9.1 Legal and ethical constraints

- **C-1** The system shall be run only on networks and hosts the team owns or is explicitly authorised to monitor.
- **C-2** Attack traffic for testing shall be generated only against the team's own targets, or replayed from capture files.
- **C-3** Public threat feeds are used under their providers' terms (abuse.ch, Spamhaus); feed data is not committed to the repository.

9.2 Out of scope

- Blocking, dropping or rewriting traffic (no IPS mode).
- Decrypting TLS; HTTP/2, HTTP/3 and QUIC payload parsing.
- Full IP fragment reassembly (fragments are tracked, not reassembled).
- Reassembly of HTTP bodies and bulk data.
- Non-Ethernet link types.

9.3 Known limitations

Area	Limitation
Capture point	Handshake-, reply- and stream-based detectors assume both directions are visible; a one-direction capture makes busy clients look like scanners.
Slow attacks	Scans, floods and brute force kept below their thresholds are not detected; the baseline can be raised slowly by a patient attacker.

Area	Limitation
ARP	If the attacker is the first host seen claiming an address, no change is visible; static bindings for the gateway are required.
TTL anomaly	Anycast and load balancers legitimately vary hop counts, so this detector is low severity.
UDP floods	Floods the target answers are not flagged by the UDP flood detector (reflection rules cover the common cases).
Beaconing	Needs at least ~10 regular connections; heavy jitter lowers detection.
JA3	Unstable for modern browsers, which randomise extension order; useful mainly for non-browser clients.
Stream reassembly	Messages that never complete are never parsed; evicted flows lose reassembly.
Baseline	Intervals close only when packets arrive; seasonality (time of day) is not modelled.
WSL2 development host	DNS goes over loopback; the clock can jump; virtual NIC offloads alter checksums and frame sizes. The final demonstration uses native Linux or a VM with offloads disabled.

10. Verification and Traceability

10.1 Verification methods

Method	Verifies	How
Unit tests (race detector on)	FR-CAP, FR-L2L3, FR-L4, FR-STR, FR-APP, FR-RULE, FR-ENG, FR-LOG, FR-QRY	Table-driven tests with hand-built frames, including malformed and boundary cases
Oracle cross-checks	FR-L2L3, FR-L4, FR-APP-2	Decoded fields compared against an independent decoder in tests only

Method	Verifies	How
Scenario harness	FR-DET, FR-COR, FR-TI	Generated captures with expected and forbidden alerts, run through the real pipeline (`make scenarios`)
Mutation checks	All detection logic	Deliberately broken logic must fail a test or scenario
Fuzzing	NFR-REL-2	Go native fuzzing of parsers, rule loader, feeds, JA3 and the full pipeline
False-positive corpus	NFR-ACC-1	~80 public benign and attack captures plus own recordings, every alert classified
Live checks	IF-CLI, IF-UI, FR-DET	Real traffic on loopback and the network interface (scans, floods, slowloris, AXFR, web attacks, DNS tunnelling)
Benchmarks and soak	NFR-PERF, NFR-MEM, NFR-REL-1, NFR-CAP-1	1M-packet replays, 1M spoofed sources, 60-minute soak with periodic attack bursts

10.2 Syllabus traceability

Syllabus topic	Requirements
Topic 3: Framing and error detection	FR-L2L3-1, FR-L2L3-2, FR-L2L3-3 (IPv4 header checksum), FR-L4-4 (TCP/UDP/ICMP checksums), FR-CAP-3 (captured vs wire length)
Topic 5: IP addressing	FR-L2L3-3 to FR-L2L3-6, FR-L2L3-5 (ARP), FR-RULE-2 (CIDRs, variables), FR-DET-8 to FR-DET-13 (Land, Smurf, fragments, TTL, ARP spoofing), FR-TI-2
Topic 6: TCP/UDP	FR-L4-1 to FR-L4-3, FR-STR (reassembly, sequence numbers), FR-DET-1 to FR-DET-7 (handshake, scans, floods), FR-DET-14, FR-DET-15

Syllabus topic	Requirements
Topic 7: Application layer	FR-APP (DNS, HTTP, FTP, TLS), FR-DET-16 to FR-DET-26 (web attacks, DNS abuse, tunnelling, credentials, beaconing)